

Trabalho Prático 2 – Ordenação em memória externa

Disciplina Estrutura de Dados

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@gmail.com

1. Introdução

A proposta do TP foi implementar um programa que ordena URL's de acordo com seu número de visitas (popularidade) utilizando a memória principal e a secundária. Foi sugerida uma implementação em duas partes: primeiro são guardados os dados ordenados (utilizando quicksort) em arquivos chamados “rodadas-n.txt” (em que n é o índice do arquivo), na segunda parte os dados dos arquivos gerados são guardados em um heap, na qual os urls e popularidade são armazenados e retirados em ordem para serem escritos no arquivo de saída. Caso haja empate na popularidade, foi considerada a ordem alfabética para realizar a ordenação.

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas de dados e os métodos usados para solucionar o problema, analisar a complexidade de tempo e de espaço do código e apresentar os resultados de análises experimentais em relação ao desempenho computacional e à localidade de referência.

Como solicitado no enunciado do TP, foi usado o método quicksort para ordenar os urls nos arquivos rodadas. Os dados a serem ordenados são passados por um arquivo de texto, que é passado como parâmetro na linha de comando. O programa também gera os dados para a análise experimental a partir da biblioteca ‘memlog’, e os erros do programa são tratados com as macros definidas em ‘msgassert.h’.

2. Método

A implementação do programa teve como base a classe TipoOrdena, que compreende a chave a ser ordenada(int), que no caso é a popularidade do site, a url (char[300]) e o número do arquivo que foi tirado a url e sua popularidade. Foi escolhida essa estrutura de dados pois com ela é possível guardar os dados em um array de TipoOrdena, facilitando a modularização dos dados e consequentemente facilitando sua ordenação.

No código são usados dois tipos de ordenação, o quicksort e a construção de um heap.

Para realizar o quicksort são utilizadas as seguintes funções:

void QuickSort(TipoOrdena, int) – Essa função chama a função Ordena. Ela serve para diminuir o número de parâmetros a serem passados para realizar o quicksort. Se fosse chamar a função Ordena diretamente seria necessário passar três parâmetros ao invés de dois.

void Ordena (int, int, TipoOrdena) – divide o vetor recursivamente e depois chama a função void particao para ordenar e combinar os subvetores.

`void Particao (int, int, int, int, Tipo ordena)` – ordena os subvetores, caso haja empate, ordena em ordem alfabética.

O pivô escolhido para o quicksort foi o elemento do meio do vetor, isso pois o pior caso do quicksort somente ocorrerá se for escolhido sistematicamente o pivô como um dos extremos do arquivo. Visto como os dados são divididos em diversos arquivos (fitas), o tamanho de cada arquivo é consideravelmente reduzido, e, portanto, mesmo que se caia no pior caso assintótico, como o vetor já foi dividido nos arquivos, então o pior caso assintótico seria amortecido. Além disso, as URL's passadas no arquivo de entrada tendem a estarem aleatoriamente distribuídas, o que dificulta o pior caso.

Para criar e manipular o heap são usadas as seguintes funções:

`void Constroi (TipoOrdena, int)` – chama a função refaz para a metade do vetor em que se o vetor fosse transformado em uma árvore binária, essa metade do vetor seriam os nós que teriam filhos.

`void Refaz (int, int, TipoOrdena)` – ordena o vetor seguindo a condição de cada nó tem que ser maior ou igual aos seus filhos, de forma que a maior chave estará na raiz, caso haja empate, ordena em ordem alfabética.

`Void RetiraMax(TipoOrdena, int)` – essa função retira o maior elemento do heap, que está na primeira posição no heap. Isso é feito de forma que o último elemento é colocado na primeira posição, então diminui o heap em uma unidade e então reordena o heap.

Já na função `main(int argc, char ** argv)`, primeiro são coletados da linha de comando o nome do arquivo de entrada, o nome do arquivo de saída, o número de entidades que cabem na memória primária e o número de fitas, o arquivo para desempenho computacional e a opção de ativar o registro de acesso.

No main foram feitos três loops principais:

No primeiro loop são lidas as entidades do arquivo de entrada e os dados são armazenados em um array da classe `TipoOrdena`, em que a chave corresponde à popularidade do site. O array é ordenado em ordem decrescente (caso haja empate, foi ordenado em ordem alfabética) através do quicksort e então os dados do array são escritos no arquivo `rodada-n.txt`, em que `n` é o número do arquivo gerado. Finalmente o arquivo gerado é fechado. Isso é feito até serem gerados as `n` fitas, cujo `n` foi passado por parâmetro na linha de comando.

No segundo loop os arquivos gerados são abertos e é colocado o primeiro elemento de cada arquivo em um heap, o heap é um array da classe `TipoOrdena`, em que a chave é a popularidade do url e também é armazenado o arquivo de onde veio esse url. Isso é feito para todos os arquivos gerados no primeiro loop.

Já no terceiro loop, o código escreve o primeiro elemento do heap no arquivo de saída, identifica de qual arquivo esse elemento veio, se o arquivo não tiver vazio é tirado outro elemento desse arquivo e trocado pelo primeiro elemento do heap e o heap é reordenado. Se o arquivo tiver vazio, ele chama a função `RetiraMax`, e retira o primeiro elemento do heap sem reposição, consequentemente, diminuindo o heap em uma unidade e reordenando ele. Isso é feito até o heap ficar vazio.

3. Análise de Complexidade

Tempo:

No main é possível ver que no primeiro loop a complexidade assintótica é $O(n)$, pois nesse primeiro loop é lido e escrito nos arquivos a mesma quantidade de elementos (site com sua popularidade) que existem no arquivo de entrada. Além disso, o quicksort tem complexidade assintótica em seu caso médio de $O(n \log n)$. No pior caso o quicksort seria chamado diversas vezes, mas para ordenar vetores pequenos.

O segundo loop principal possui complexidade assintótica $O(n)$.

Em seguida é chamada a função Constrói para o heap que na tabela abaixo é possível ver que sua complexidade é $O(n \log n)$.

Então no último loop principal, há a possibilidade de ser chamada a função Refaz ou a função RetiraMax, e ambas são $O(\log n)$, o que faz esse loop ser $O(\log n)$. No pior caso, essas funções seriam chamadas para cada elemento provido na entrada, o que pode ser bem custoso, mas em média o custo dessas duas funções dentro desse loop seria $O(n \log n)$.

Além disso são escritas na saída as entidades ordenadas, o que é $O(n)$, então o segundo loop é $O(n)$.

Já no último loop para fechar os arquivos a complexidade é $O(n)$.

	Constrói	Insere	Retira máximo	Altera prioridade
Heaps	$O(N \log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$

Portanto, a complexidade do código é $O(n)$.

Espaço:

No código, além de outras variáveis que ocupam $O(1)$ de espaço, foram usadas duas arrays da estrutura de dados TipoOrdena, uma dessas arrays tem o tamanho do número de entidades que devem ser lidas para cada arquivo, e a outra tem o tamanho do número de fitas. Além disso, no código é alocado espaço para os arquivos rodadas, que no total ocupam no máximo o mesmo espaço do arquivo de entrada, ou seja, o conjunto desses arquivos é $O(n)$. Dessa maneira, o código tem complexidade de espaço $O(n)$.

1.4 Análise Experimental

Desempenho Computacional

As métricas importantes para desempenho computacional da ordenação são as comparações e ordenações dentro das funções Particao e Refaz. O código será analisado em duas partes, a primeira parte do algoritmo corresponde somente ao primeiro loop principal do código (onde é realizado o quicksort), já a segunda parte corresponde aos dois próximos loops principais (onde é montado o heap e escrito o arquivo de saída).

O vetor inicial usado para fazer a análise foi um vetor aleatório e para cada valor calculado foi feita uma média com dez experimentos, os valores da tabela estão em milissegundos

e foi medido pela biblioteca <chrono>. O número de rodadas é sempre igual ao número de fitas, mas o tamanho do vetor inicial muda, se forem 10 fitas e 10 entidades, o tamanho será 100, se forem 20, então o tamanho será 400 e assim por diante.

Fitas x Entidades	10 x 10	20 x 20	30 x 30	40 x 40	50 x 50
Quicksort	0,4091962	0,8245252	1,3761618	2,1318853	3,1135586
Heap	0,1291572	0,4073635	0,8498186	1,4152535	2,4412781
Diferença	0,280039	0,4171617	0,5263432	0,7166318	0,6722805
Soma	0,5383534	1,2318887	2,2259804	3,5471388	5,5548367
Fitas x Entidades	60 x 60	70 x 70	80 x 80	90 x 90	100 x 100
Quicksort	4,0649945	5,7383781	6,9705334	8,2848696	9,966833
Heap	3,4632451	4,8391646	6,0258691	7,7591736	9,4463587
Diferença	0,6017494	0,8992135	0,9446643	0,525696	0,5204743
Soma	7,5282396	10,5775427	12,9964025	16,0440432	19,4131917

É importante notar que na primeira parte do código é lido a entrada e na segunda parte é escrito a saída, o que causa um amortecimento na diferença de tempo entre as duas partes, mas ainda assim a primeira parte do código é aparentemente dominante. A diferença entre as partes do código tende a crescer com exceção dos testes com entrada de tamanhos 8100 e 10000.

Isso ocorre porque o tempo da segunda parte do código tende a aumentar mais rapidamente que a primeira, pois apesar da função Refaz ter o custo assintótico menor, o heap precisa ser refeito para cada entidade tirada, já o quicksort só é chamado para cada fita, ou seja, se houver 100 elementos na entrada para ser distribuído em 10 fitas o heap será refeito muitas mais vezes que o quicksort será chamado. Portanto, à medida que o número de entidades de entrada vai aumentando, a diferença entre o tempo gasto na primeira parte e na segunda parte tende a diminuir.

Isso é provado pela tabela abaixo.

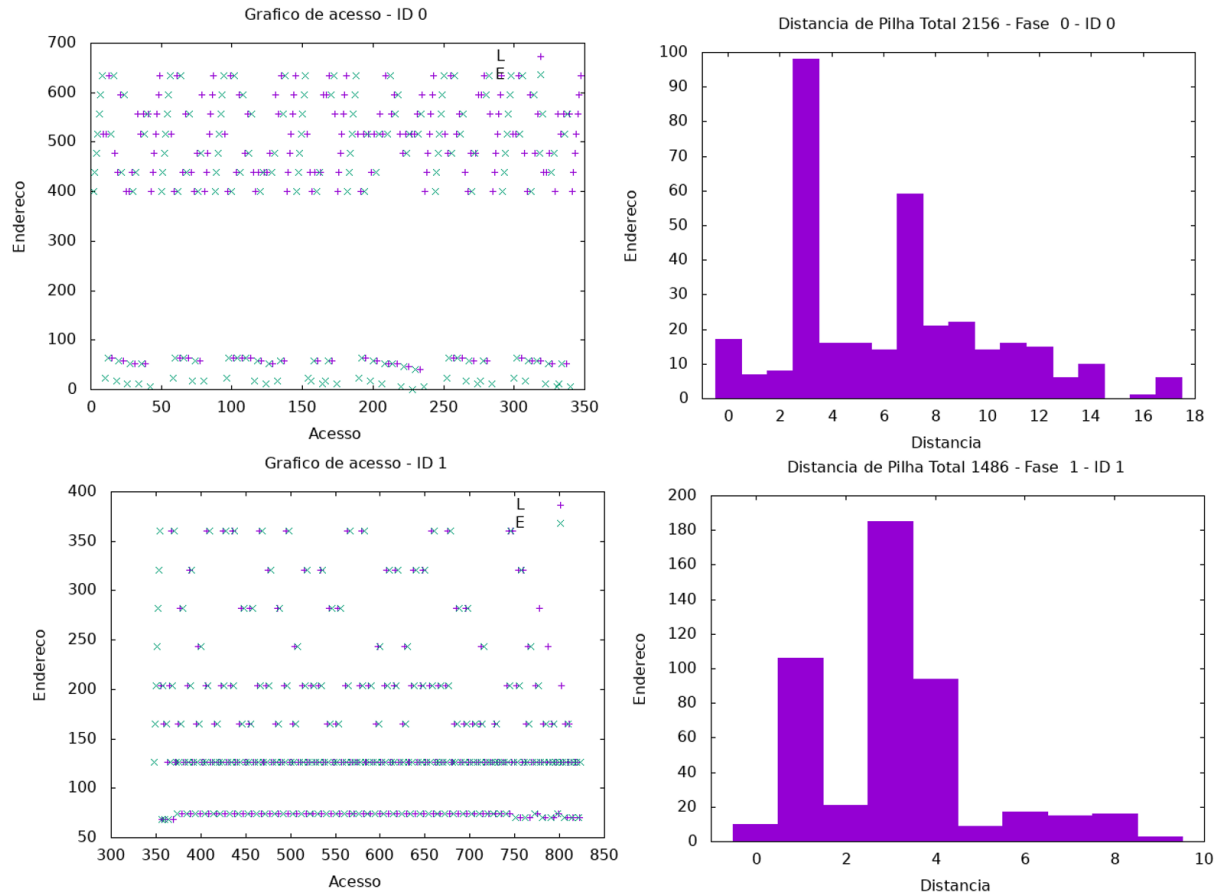
Fitas x Entidades	130x 130	140x 140	150x 150	160x 160
Quicksort	12,513054	14,220309	15,347736	16,102521
Heap	13,379338	16,436649	18,561875	18,835493
Diferença	-0,866284	-2,21634	-3,214139	-2,732972
Soma Total	25,892392	30,656958	33,909611	34,938014

Portanto, apesar do heap ser menos custoso que o quicksort, como o heap precisa ser refeito muitas vezes, ele se torna mais custoso que o quicksort.

Olhando o código como um todo, é possível ver que o crescimento dele é linear. Pois a entrada na tabela está aumentando quadraticamente assim como o tempo do código. Pegando um exemplo linear, onde a entrada tem 50 fitas, o tamanho da entrada é 2500, e onde a entrada tem 70 fitas, o tamanho da entrada é 4900, quase o dobro de 5000. Olhando o tempo total desses dois intervalos é possível ver que o tempo de execução duplicou.

Já onde a entrada tem 100 fitas, o tamanho da entrada é 10000, quase o dobro de 4900. Olhando o tempo total dos intervalos de 70 e 100 é possível ver que o tempo de saída tende a duplicar e assim por diante.

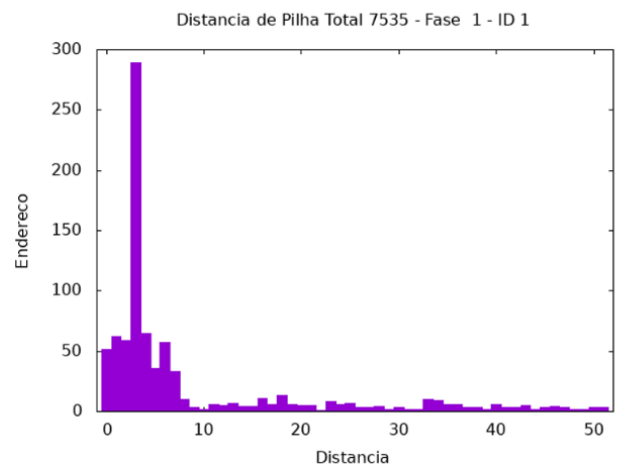
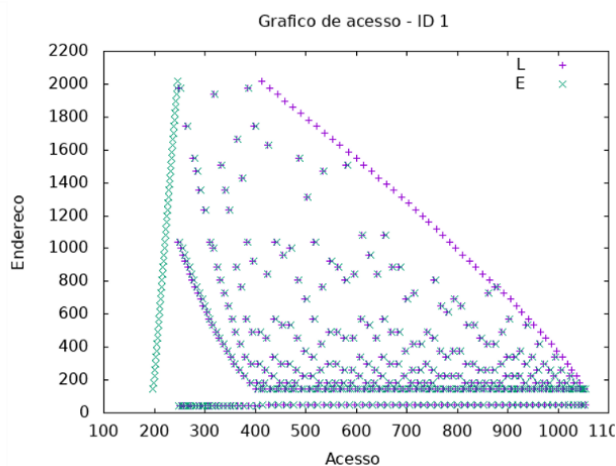
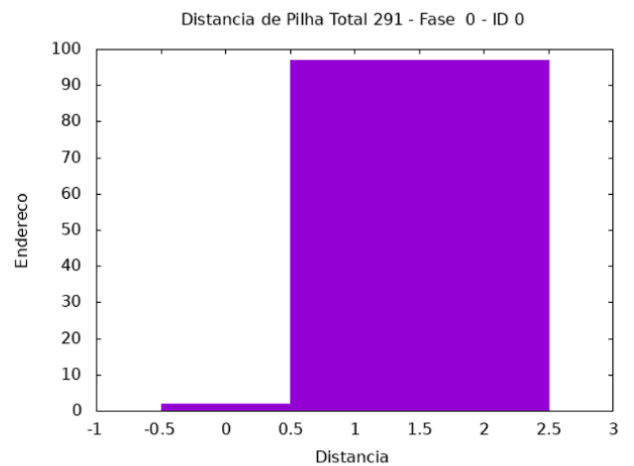
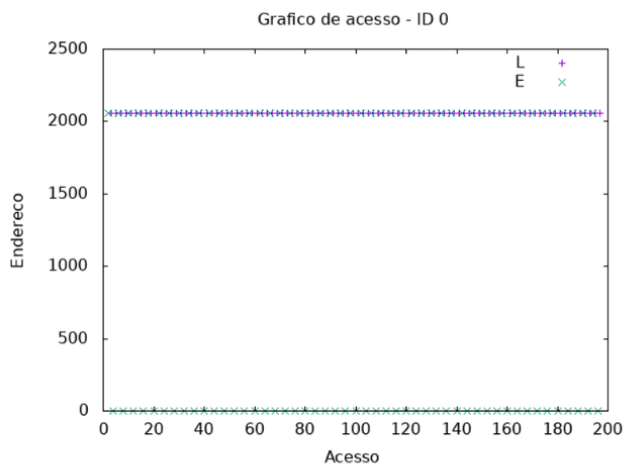
Análise de Padrão de Acesso à Memória e Localidade de Referência



A fase um representa a fase de receber a entrada, ordenar o vetor e escrever o vetor no arquivo rodada. A fase dois representa a parte em que a primeira entidade de cada arquivo rodada é escrita no heap e ordenar ele e então ele é escrito no arquivo de saída.

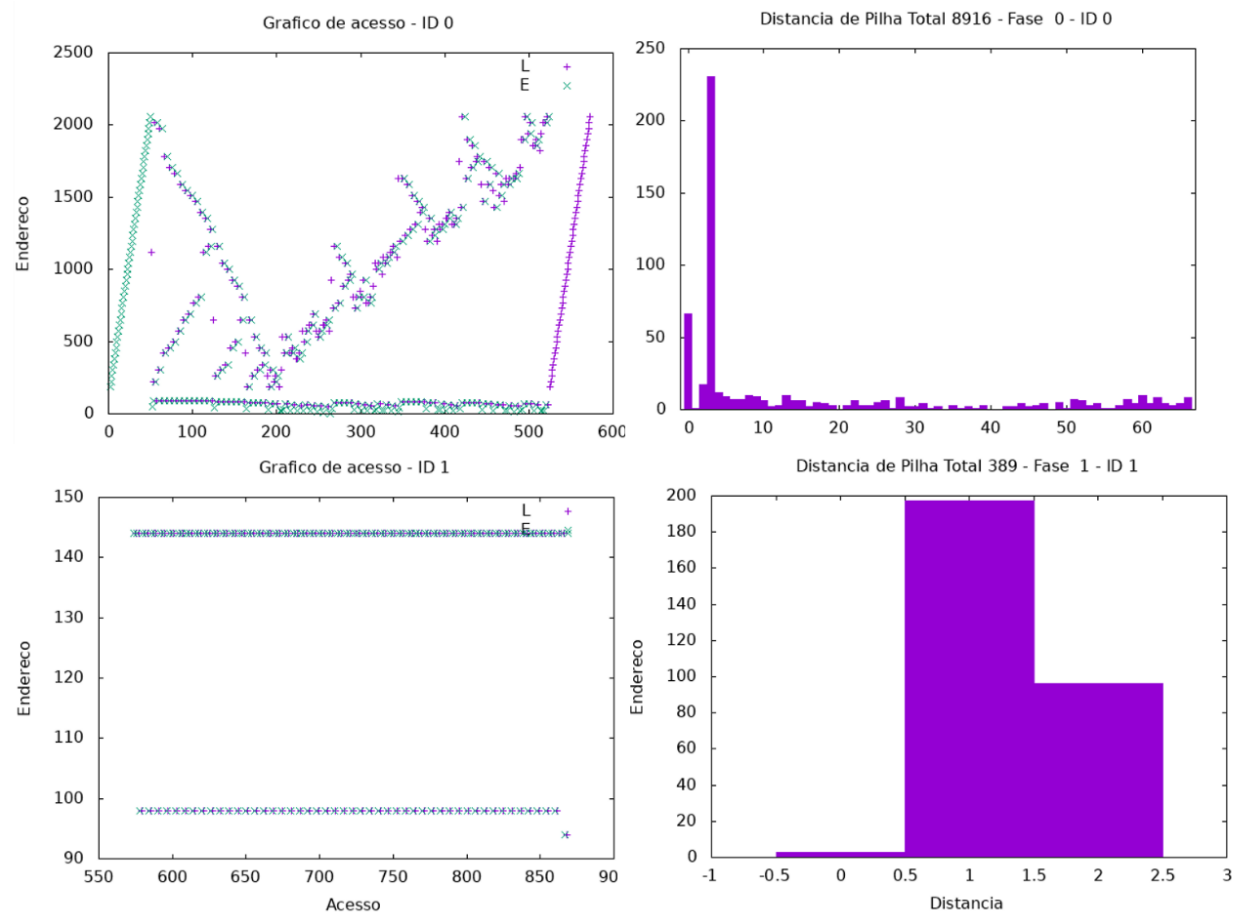
Esses gráficos acima foram feitos usando sete fitas e sete entidades para cada fita totalizando 49 entidades. No primeiro gráfico da parte de cima é possível notar que as entidades são lidas do arquivo de entrada de sete em sete e então são ordenadas usando quicksort e em seguida são escritas no seu respectivo arquivo de saída, isso é feito sete vezes. Já no gráfico de cima da direita é possível ver que a distância de pilha é grande no três, isso se dá porque o pivô do quicksort para um array de tamanho sete sempre começa no três. Já o tamanho de pilha é grande no sete porque após o array TipoOrdena ser ordenado ele é escrito no arquivo da rodada começando do índice 0.

No primeiro gráfico da parte de baixo é possível notar que é lido de cada arquivo gerado uma entidade, então a ordenação do heap a partir da função Refaz e é sempre escrita no arquivo de saída a mesma posição de memória. Já no histograma a distancia de pilha é grande no um no dois e no quatro provavelmente porque o heap é ordenado de acordo com seus filhos, então são muitas trocas em posições de memória que estão há distancias pequenas.



Esses gráficos acima foram feitos usando 49 fitas e 1 entidade para cada fita totalizando 49 entidades. No primeiro gráfico da parte de cima é possível notar que as entidades são lidas do arquivo de entrada mas não são ordenadas pelo quicksort, elas são diretamente escritas nas fitas. Já no gráfico de cima da direita é possível ver que a distancia de pilha é pequena, porque é somente uma unidade para cada fita e o quicksort não foi usado.

No primeiro gráfico da parte de baixo é possível notar que é lido de cada arquivo gerado uma entidade e que é sempre escrita no arquivo de saída a mesma posição de memória, há a primeira ordenação do vetor e então várias outras ordenações e então é escrito no arquivo de saída. Já no histograma as maiores distancias de pilha estão entre 0 e 9, e a maior delas é o 3, como visto anteriormente o heap é ordenado de acordo com seus filhos, então são muitas trocas em posições de memória que estão há distancias pequenas em relação ao total.



Esses gráficos acima foram feitos usando 1 fita e 49 entidades para a fita totalizando 49 entidades. No primeiro gráfico da parte de cima é possível notar que as entidades são lidas do arquivo de entrada e são ordenadas pelo quicksort, e então são todas escritas contiguamente. Já no gráfico de cima da direita é possível ver que a maior distancia de pilha é o três, como o quicksort é chamado somente uma vez, não teria sentido nesse caso o 24(pivô) ter uma distância de pilha grande.

No primeiro gráfico da parte de baixo é possível notar que é lido de cada arquivo gerado uma e então ela é imediatamente escrita no arquivo de saída, sem precisar de ordenação, pois há somente uma entidade em cada arquivo rodada e as entidades já estão ordenadas nesses arquivos. É notável que o histograma possui uma distancia de pilha total baixa, pois não houve ordenação.

1.6 Conclusões

A proposta desse trabalho foi realizar a implementação do ordenador de urls usando a memória principal e secundária. Foi um desafio fazer esse trabalho e implementar os métodos de ordenação. Aprendi sobre como implementar o quicksort e como fazer um heap. O trabalho também trouxe diversos conhecimentos sobre manipulação de arquivos e sobre criação a manipulação de classes em array.

Na parte de desempenho computacional tive dificuldades em usar o gprof pois seria necessária uma entrada absurdamente grande para ter um resultado satisfatório. Então eu pesquisei e aprendi sobre a biblioteca <chrono>, que possibilitou a realização da análise

de desempenho computacional com uma entrada bem menor e apesar de ele não gerar a mesma quantidade de informações que o gprof, ele foi muito mais preciso e consistente.

1.7 Bibliografia

CHAIMOWICS, L and Prates. Slides virtuais e vídeos da disciplina de estrutura de dados. Disponibilizados via moodle nas seções Estrutura de Dados (listas lineares), e Algoritmos de Ordenação (quicksort e heapsort).

<https://pt.stackoverflow.com/questions/278269/como-que-eu-posso-medir-o-tempo-de-execu%C3%A7%C3%A3o-de-um-programa-em-c>

1.8 Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Acesse o diretório TP. Compile os arquivos usando o comando make. O executável gerado estará em bin. Agora entre no diretório bin e execute o programa.

Para executar o programa deve-se passar na linha de comando o arquivo com os dados para ordenação, o arquivo de saída, o número de entidades que cabem na memória principal, o número de arquivos fita a serem gerados, o nome do arquivo onde será feito o registro de acesso. A última opção de argumento na linha de parâmetro é facultativa, escreva 'l' caso desejar que seja registrado os acessos à memória, não escreva nada caso quiser que seja registrado somente o tempo de execução.

O arquivo de entrada passado pela linha de comando deve estar no diretório bin.

Por exemplo: `”./main entrada.txt saída.txt 10 5 analise.txt l”`

O arquivo de saída seria o “saída.txt” e será registrado os acessos à memória no arquivo analise.txt. Seriam geradas 5 fitas com 10 linhas cada.